Deep Learning Optimized on Jean Zay

Dataset optimization

Storage spaces and data format

Authors: Bertrand Cabot, Myriam Peyrounette

Commented slides

Last update: March 2026

Dataset optimization

Main bottlenecks ◂

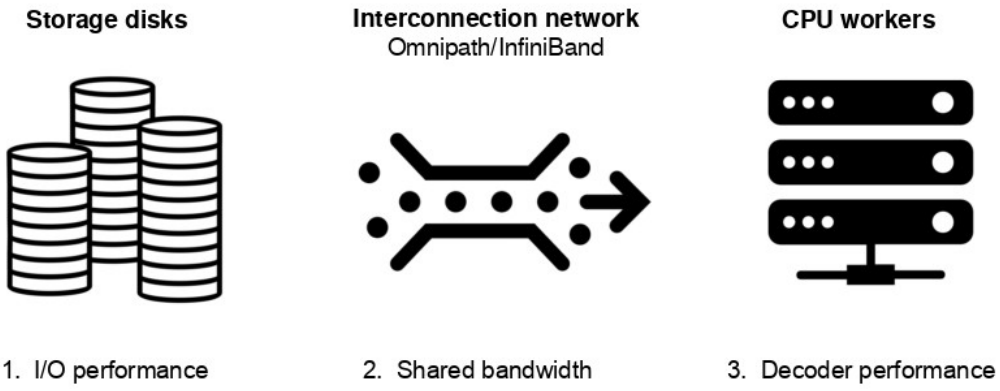
Data storage – various disk spaces ◂

Data format – at sample level ◂

Data format – at dataset level ◂

2

Bottlenecks upstream of DataLoader

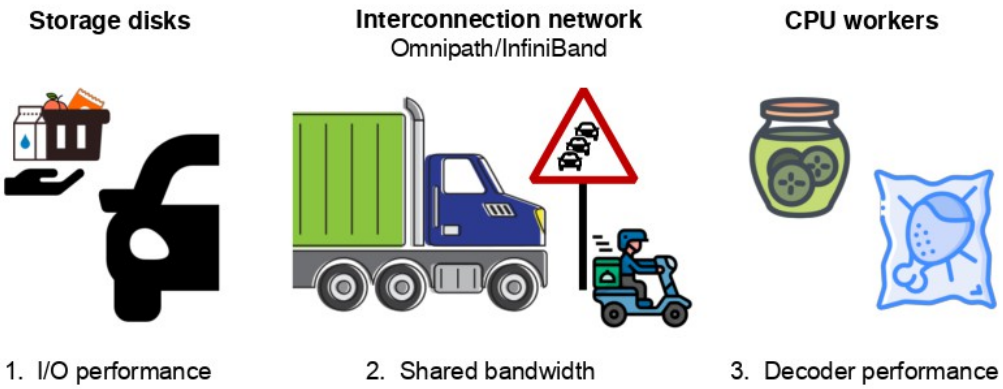


3

In this section, we will present different optimization possibilities in regard to the input data format.

We are seeking to avoid three main obstacles: Slow I/O read speed on disks, bandwidth sharing in the interconnection bus and the cost of input data decoding on CPU.

Bottlenecks upstream of DataLoader



4

To understand the various bottlenecks encountered, here is an analogy: reading data from a storage disk is like picking up your groceries at a drive-through.

Once at the drive-through, the staff will be more or less quick to serve you (analogy with the performance of different types of storage disks).

On the way back, there may be more or fewer people with you on the road, and even possible traffic jams (analogy with bandwidth sharing on the interconnect bus).
If you collect few products at a time and travel by scooter, you will suffer less from traffic jams but will have to make more trips back and forth.
If you collect a lot of products at a time and travel by semi-trailer, you will suffer from traffic jams but will not have to return to the drive-through for a while.

Once at home, you will eat more or less quickly depending on the preparation time required to cook the type of food you have chosen (analogy with the more or less transformed data format).

Dataset optimization

Main bottlenecks ◂

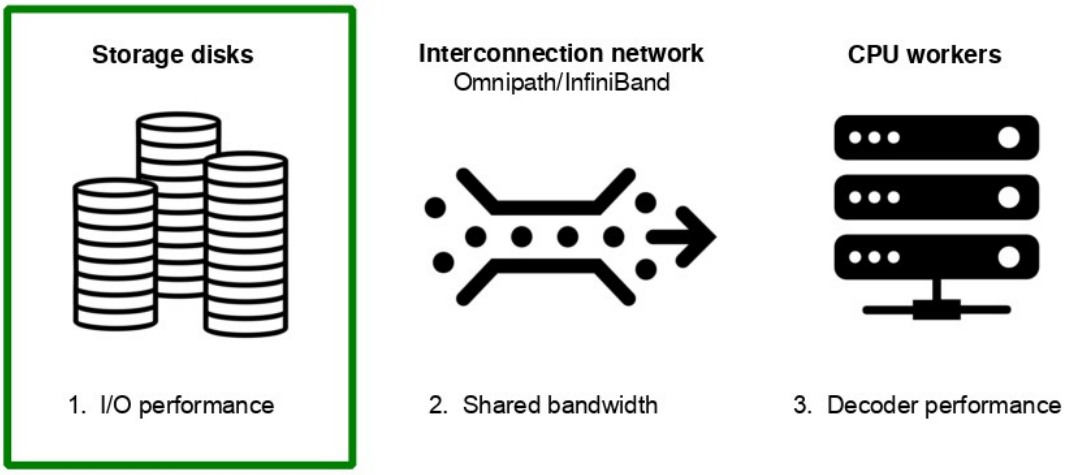
Data storage – various disk spaces ◂

Data format – at sample level ◂

Data format – at dataset level ◂

4

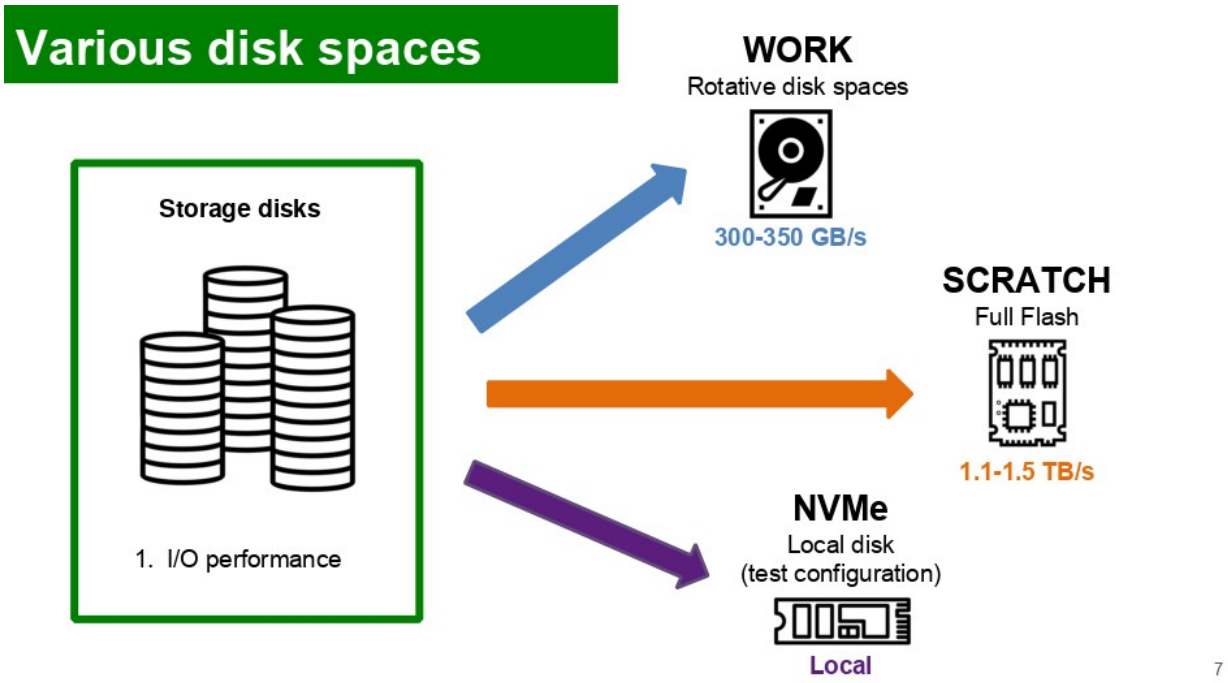
Bottlenecks upstream of DataLoader



Where should I store my dataset?

6

There are different storage technologies on Jean Zay. Where is it pertinent to store your input data?



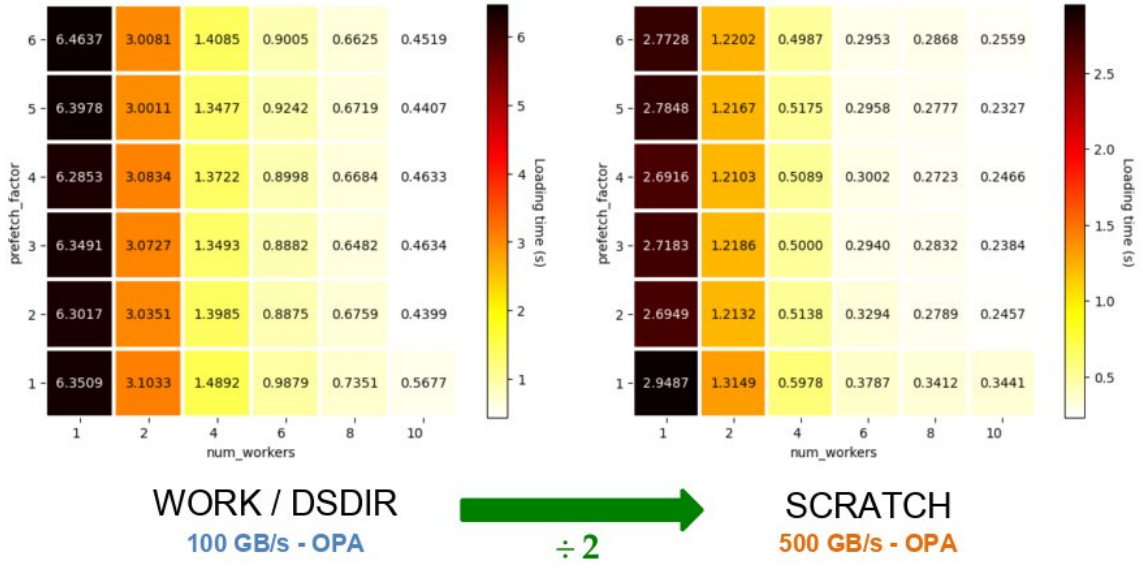
The WORK space store information on rotative disks with a read speed of 350 GB/s (300 GB/s in writing).

The SCRATCH stores information on a Full Flash (SSD) disk with a read speed of 1.5 TB/s (1.1 TB/s in writing), which is 5 times faster than with rotative disks.

It is possible to connect local disks (NVMe) directly to a compute node but this technology is not manageable at the level of a supercomputer and is only available internally for tests of technology watch.

Various disk spaces

dlojz.py - 50 iterations - test partition gpu_p4



NOTE: Test performed in 2023, when Jean Zay had a GPFS parallel file system.

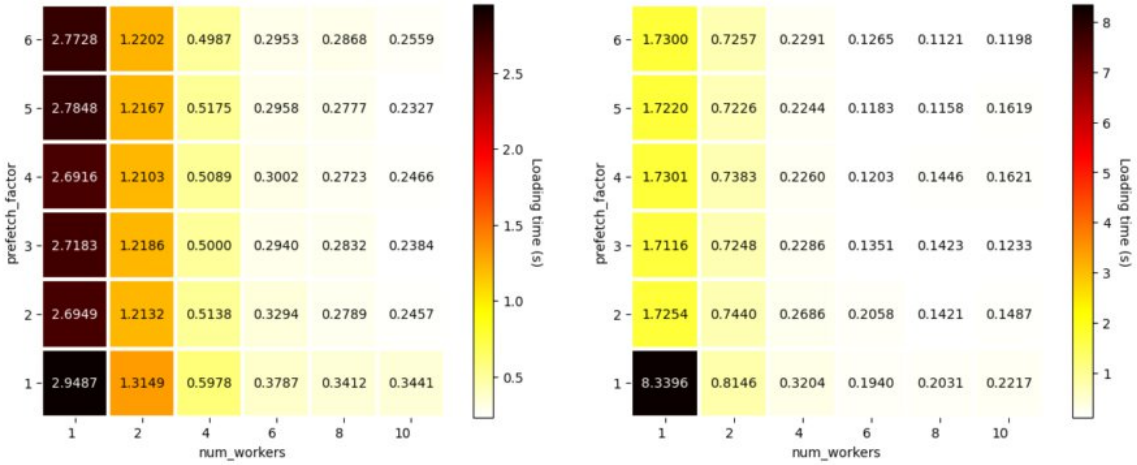
Here we ran the `dlojz.py` script during 50 iterations on a A100 PCIe GPU (test partition equipped with NVMe local disks).

We see the loading time (`DataLoader`) in relation to the storage technology used, the number of workers and the `prefetch_factor`.

We observe that the time is divided in half when we store the database on the SCRATCH rather than on the WORK.

Various disk spaces

dlojz.py - 50 iterations - test partition gpu_p4



SCRATCH
500 GB/s - OPA

÷ 2

NVMe
PCIe

We observe that the time is divided by 2 when we store the database on NVMe rather than on the SCRATCH.

Various disk spaces

- **NVMe**
  - ✓ Best IO performance
  - – You need to copy your dataset on the local disk first, which can take a very long time
  - – This solution is not suitable at the scale of a supercomputer so it is not available to users
- **SCRATCH**
  - ✓ Second best IO performance
  - ✓ Very large quota (bytes and inodes)
  - – 30 days file lifespan
- **WORK**
  - – Worst performance (but not bad)
  - – Only 5 TB and 500k inodes
  - ✓ No automatic deletion procedure

10

The advantages and disadvantages of the different storage spaces.

Various disk spaces

- **What about the STORE?**
  - Magnetic tape storage (with a cache on rotative disk)
  - 50 TB and 100k inodes
  - Intended to receive few very large-sized files (up to 10 TiB per file) → **archives**
  - Not accessible from the compute nodes (very high read/write latency)
- **Good practice**
  - If you don't need IO performance (when building your workflow, debugging, or simply if IO is not a bottleneck), store your dataset in the **WORK** if it is not too large
  - If you need IO performance or if it is too large, store your dataset in the **SCRATCH**
  - Keep a duplicate of your dataset **as an archive in the STORE**

11

The STORE is suitable for storing a copy of your database in archive form.

Dataset optimization

Main bottlenecks ◂

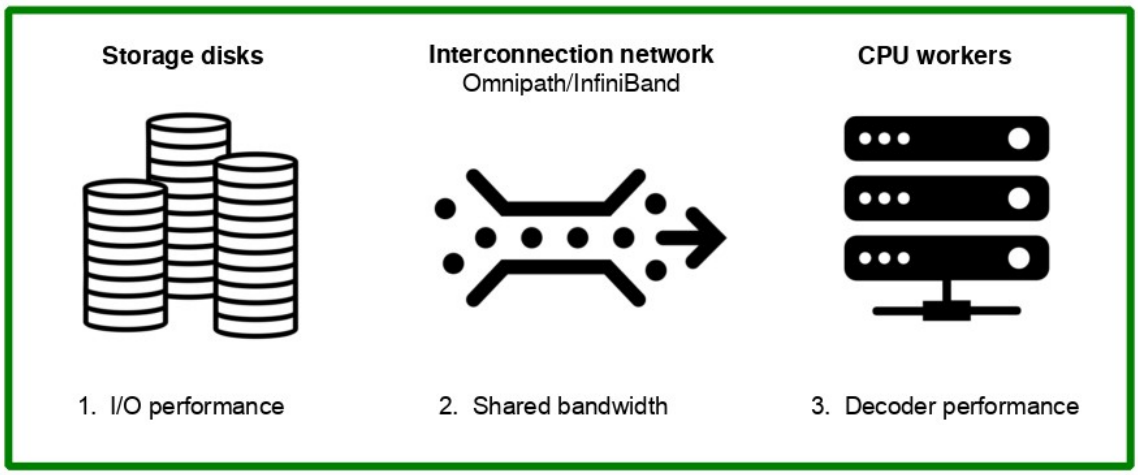
Data storage – various disk spaces ◂

Data format – at sample level ◂

Data format – at dataset level ◂

10

Bottlenecks upstream of DataLoader



Which format for my data?

13

The input data format will have an impact on all the steps.

At sample level - Sample decoding

Binary format: Pickle format, HDF5,...
Decoded more quickly, takes more space

- ✓ Decoder performance
- – Shared bandwidth
- – Storage volume

Compressed format: jpeg, png,...
Decoded more slowly, takes less space

- – Decoder performance
- ✓ Shared bandwidth
- ✓ Storage volume

The binary formats enable a rapid decoding on CPU but use more disk space (~10x) and, therefore, more bandwidth.

The compressed formats reduce the necessary storage volume and use less bandwidth but the decoding time on CPU is longer.

Dataset optimisation

Main bottlenecks ◂

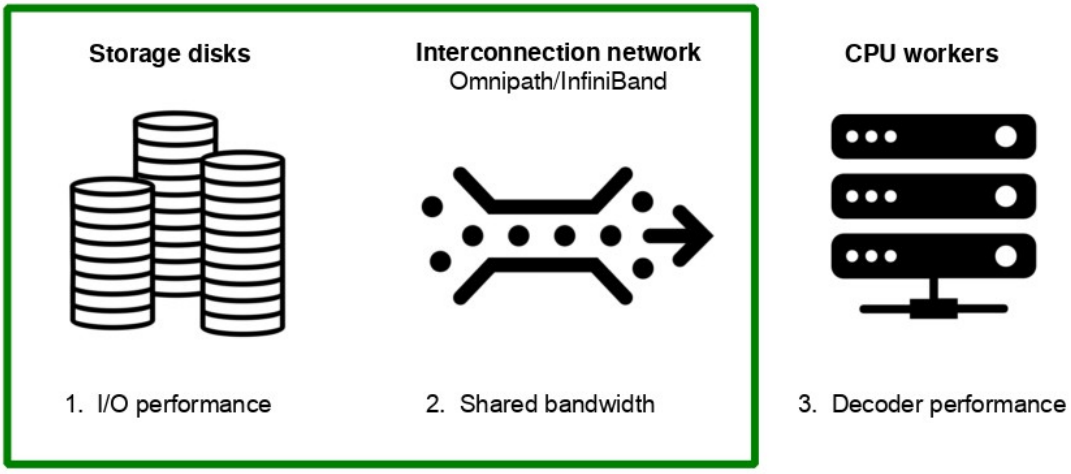
Data storage – various disk spaces ◂

Data format – at sample level ◂

Data format – at dataset level ◂

13

Bottlenecks upstream of DataLoader

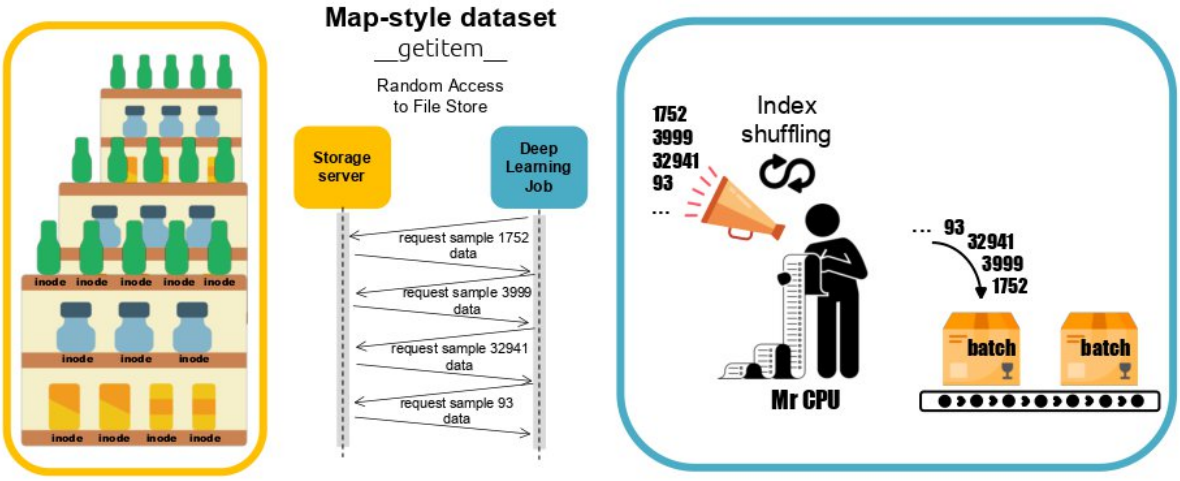


Which format for my dataset?

16

Which format should I use for my database?

Intuitive way



Pros: Easy to handle, random access possible
Cons: Lots of inodes, lots of I/Os

Here: 1 image = 1 file.

A map-style dataset corresponds to a dictionary (idx, image).

We access one data at a time from its index.

We generate one I/O request per each accessed data.

The shuffling is carried out on the indexes of the whole dataset before creating the batches.

Too many inodes is an issue

Error: Disk quota exceeded 😱

- $WORK quota per user is 5 TB / **500 kinodes**
- $SCRATCH safety quota per user is ~500TB / **150 Minodes**

+ Parallel file system does not like:

- intensive IO workloads involving lots of small files
- large number of files in a single directory

18

Reminder of the inode quotas on the Jean Zay disk spaces.

Originally, parallel file systems are designed to optimize parallel access to the same large file. It is adapting to new practices but is still suboptimal for I/O-intensive workloads on small-size files. Furthermore, the stuggle managing folders containing a very large number of files.

Too many inodes is an issue

- IDRIS recommendation on Jean Zay: **less than 100k inodes per folder**
- Repository limitations and recommendations on HuggingFace Hub 🤗

Characteristics	Recommended	Tips
Files per repo	<100k	merge data into fewer files
Entries per folder	<10k	use subdirectories in repo

https://huggingface.co/docs/hub/storage-limits#repository-limitations-and-recommendations

+ for large datasets, **WebDataset** format is recommended

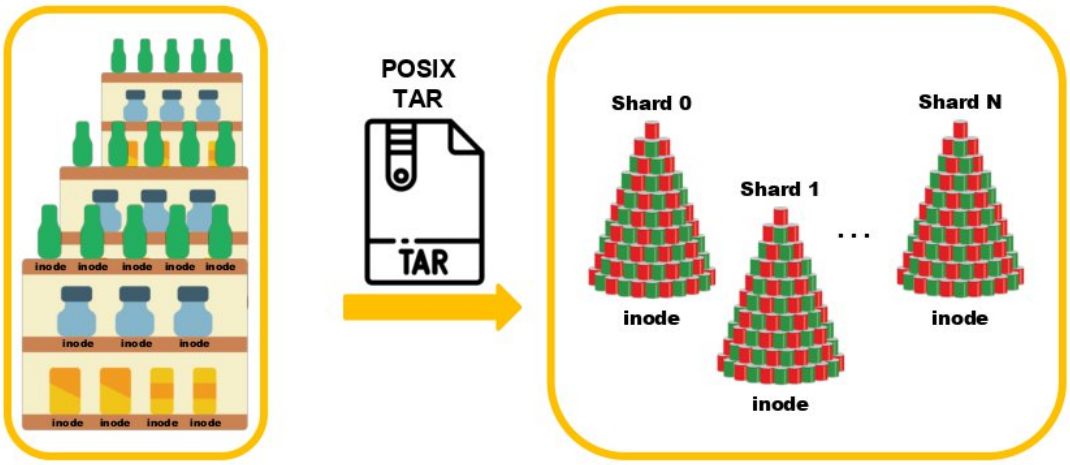
19

On Jean Zay, IDRIS recommends not storing more than 100k inodes per directory so as not to overload the parallel file system.

HuggingFace Hub also provides recommendations on dataset storage. The platform's success has led to a standardization of dataset formats that is worth following.

For large datasets, it is recommended to gather (archive) several samples in a single file. The Webdataset format is recommended.

WebDataset format – Gathering inodes

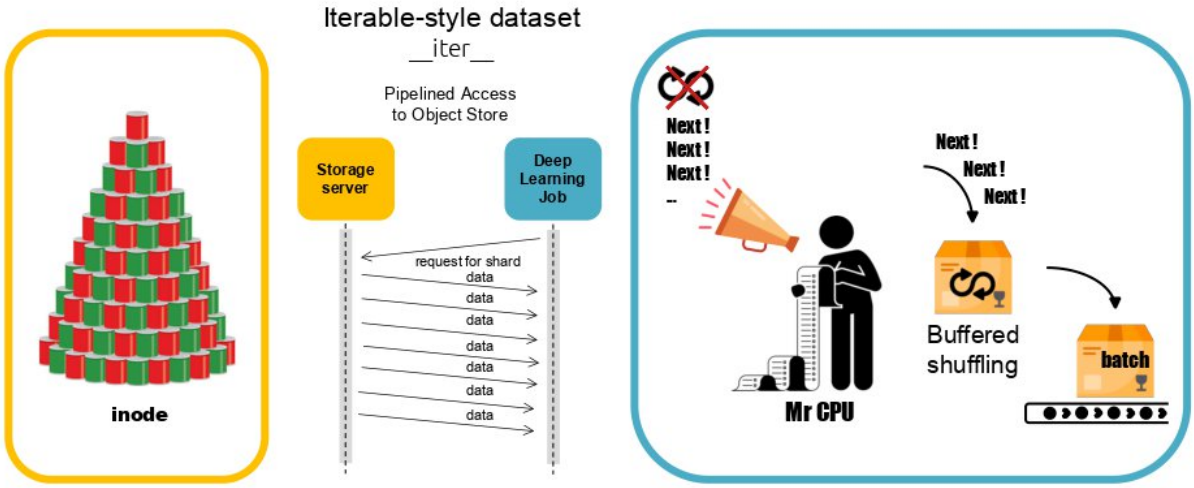


17

We present the WebDataset format in order to minimize the number of inodes.

The idea is to group the data in archive form.

WebDataset format – Iterable dataset



Pros: Fewer I/Os, fewer inodes
Cons: Difficult to shuffle or distribute, unknown dataset length

18

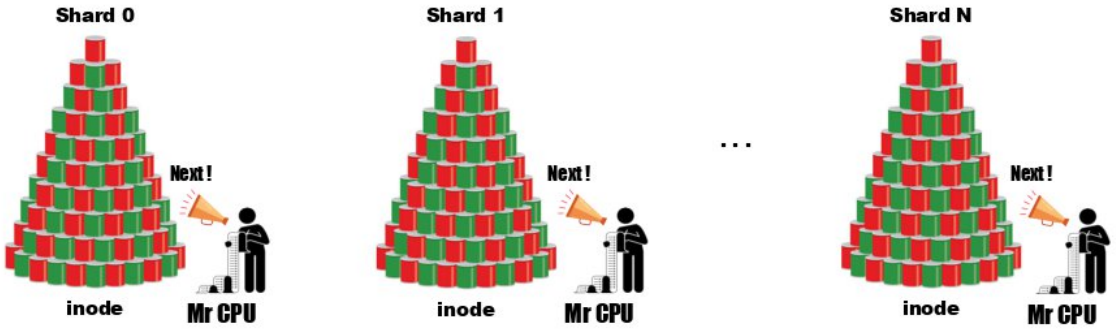
WebDataset is an iterable-style dataset. We access data contiguously. There is no mapping (idx, image).

This style of dataset is adapted to databases whose structure is not known beforehand. This is the case here because the data are « hidden » in the archives.

Shuffling is managed by the CPU at the level of a buffer. The size of this buffer is chosen by the user (proportionate to the batch size, for example).

WebDataset format - Sharding

Sharding is necessary to benefit from parallel implementation
(DataLoader multi-processing and Distributed Data Parallelism).



The number of shards should be a multiple of the number of tasks/GPUs.

19

1 shard = 1 archive.

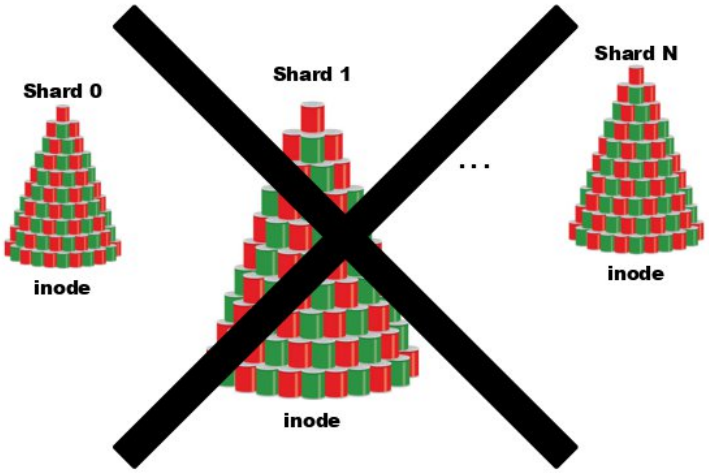
Sharding consists of dividing the original dataset on multiple archives. A minimum number of archives is necessary to generate parallelism at the I/O level.

For example, 2 DataLoader workers will access two different archives.

Likewise, 2 GPUs must be supplied by different archives.

For example: nshards = ngpus x nworkers

WebDataset format - Sharding



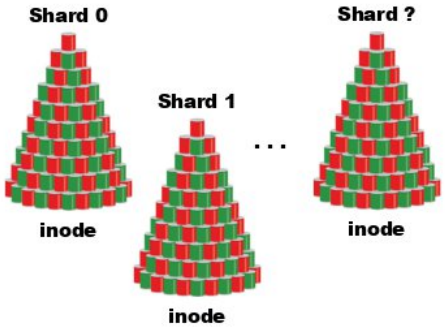
Samples must be evenly distributed among the shards to balance the workload between processes.

20

For the workload to be optimally distributed on the different processes, the data must be distributed as equally as possible in the archives.

WebDataset format - More or less shards?

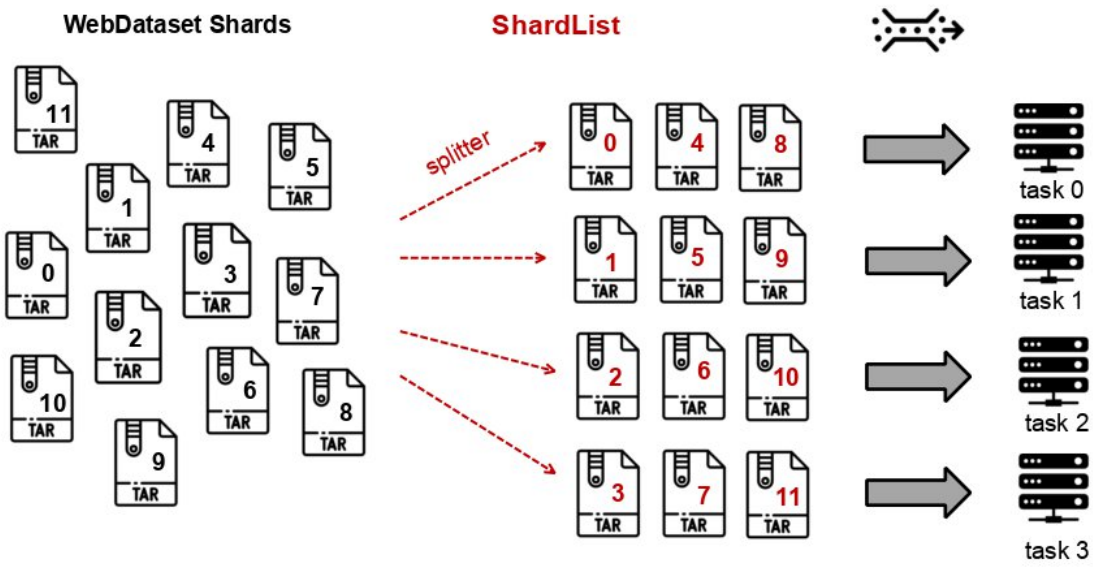
	More shards	Less shards
Large scale distribution	+	-
Shared bandwidth	+	-
Inodes quota	-	+
Number of I/O	-	+



21

The advantages and disadvantages of having a dataset with more/less shards.

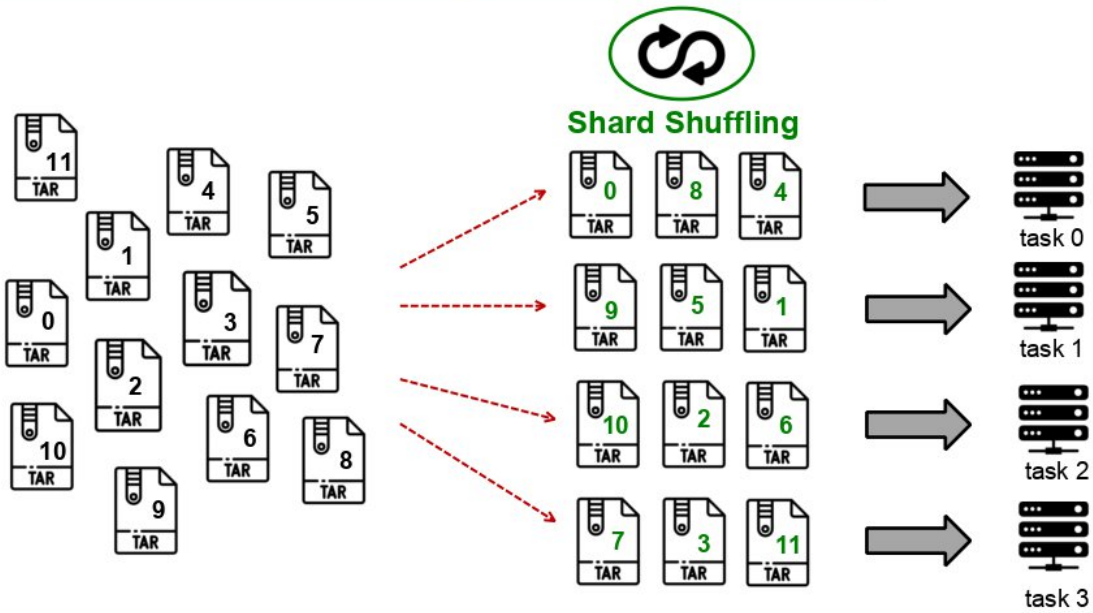
WebDataset – Multiworker sharding



22

The shards are distributed on different processes according to the splitter function.

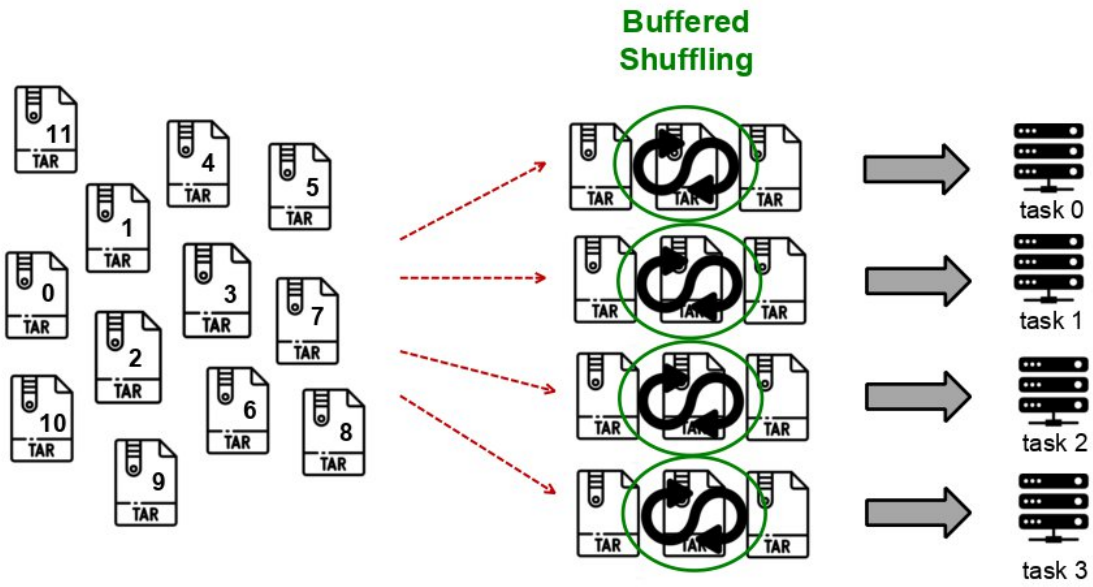
WebDataset - Shuffling



23

We can activate the first shuffling on the shard indexes. This shuffling is performed per process.

WebDataset - Shuffling



24

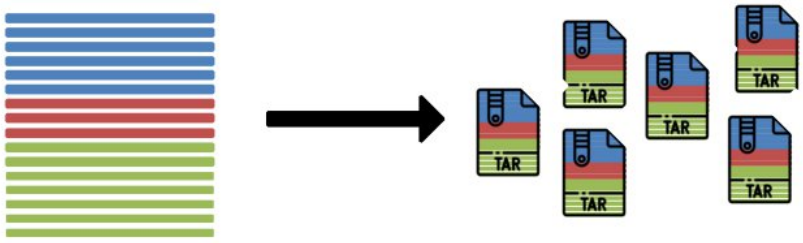
We can also activate a shuffling at the level of all the data contained on a process.

Combining the two types of shuffling (shard shuffling and buffered shuffling) is the most effective method.

WebDataset - Generation

When generating WebDataset shards, don't forget to:

- Distribute the samples as **evenly** as possible among the shards.
- Choose the number of shards **according to the number of GPUs** you will use.
- Distribute the samples so that each shard contains a **representative part of the dataset.**



+ Converting data before creating the archives to improve decoding performance?

25

Some advice for generating a dataset of the WebDataset type.

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("torchrgb")\
                                .to_tuple('input.pyd', 'output.pyd')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```


Here is an example to use a WebDataset in your script. Details are provided below step by step.

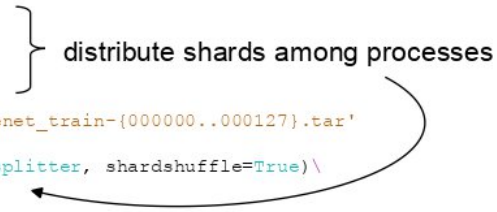
WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("torchrgb")\
                                .to_tuple('input.pyd', 'output.pyd')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```



WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                               .shuffle(1000)\
                               .decode("torchrgb")\
                               .to_tuple('input.pyd', 'output.pyd')\
                               .map_tuple(transform, lambda x: x)\
                               .batched(mini_batch_size)\
                               .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

shuffling shards indexes per process

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                               .shuffle(1000)\
                               .decode("torchrgb")\
                               .to_tuple('input.pyd', 'output.pyd')\
                               .map_tuple(transform, lambda x: x)\
                               .batched(mini_batch_size)\
                               .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

shuffling samples per process

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                               .shuffle(1000)\
                               .decode("torchrgb")\
                               .to_tuple('input.pyd', 'output.pyd')\
                               .map_tuple(transform, lambda x: x)\
                               .batched(mini_batch_size)\
                               .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

description of shard content

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                               .shuffle(1000)\
                               .decode("torchrgb")\
                               .to_tuple('input.pyd', 'output.pyd')\
                               .map_tuple(transform, lambda x: x)\
                               .batched(mini_batch_size)\
                               .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

transforming and batching

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("torchrgb")\
                                .to_tuple('input.pyd', 'output.pyd')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

define len(train_dataset)

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("torchrgb")\
                                .to_tuple('input.pyd', 'output.pyd')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

batching handled by WebDataset class

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("torchrgb")\
                                .to_tuple('input.pyd', 'output.pyd')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

usual DataLoader args

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("torchrgb")\
                                .to_tuple('input.pyd', 'output.pyd')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

drop_last equivalent

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("torchrgb")\
                                .to_tuple('input.pyd', 'output.pyd')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

define len(train_loader)

WebDataset - Perfomance test

I/O loop over the dataset
(calculation-free iterations)

```
start_time = datetime.datetime.now()

for i, (images,labels) in enumerate(loader):
    print(f'{i} / {nb_batches}', end="\r")

end_time = datetime.datetime.now()
delta_time = (end_time_it - start_time_it).total_seconds()
```

- Execution on 1 GPU

38

On a Resnet50/Imagenetcomplete training, no advantage is seen in using the WebDataset format.

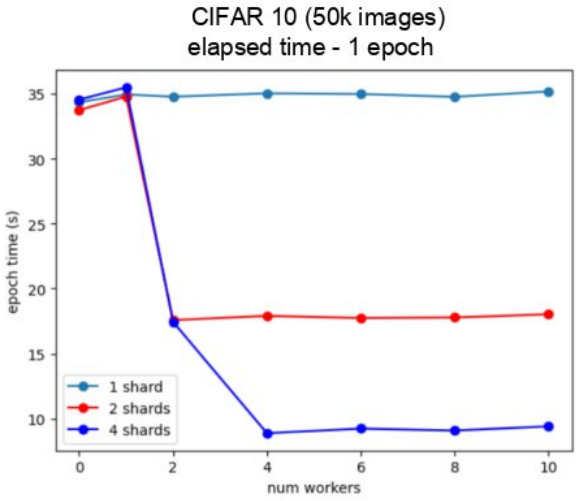
I have reduced the test to a simple dataset path to isolate the I/0 performance. This test is executed on only 1 GPU.

WebDataset - Perfomance test

I/O loop over the dataset
(calculation-free iterations)

CIFAR10 ~ 50k images

- Sharding is necessary to benefit from parallel implementation (DataLoader multi-processing).



Test on CIFAR10.

Observe that the more shards there are, the more we generate parallelism.

Here we are using a small dataset to be able to rapidly generate WebDataset versions with a higher or lower number of shards.

In reality, using WebDataset on such a small dataset deteriorates the I/O performances.

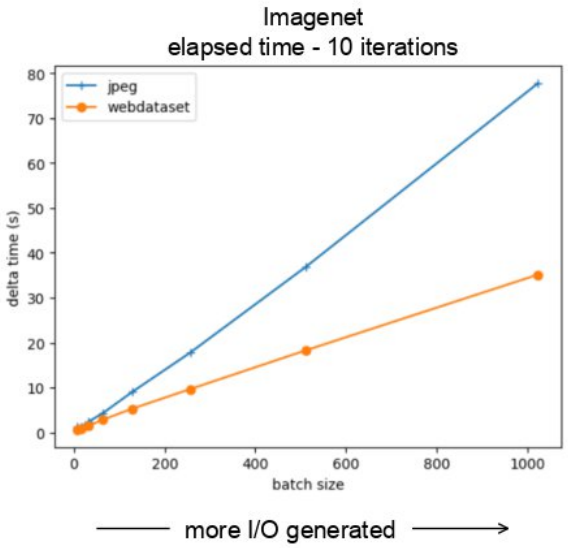
The next tests will be conducted on Imagenet.

WebDataset - Perfomance test

I/O loop over the dataset
(calculation-free iterations)

Imagenet ~ 1.3M images
128 shards ~10k images per shard (+labels)
1 shard (images + labels) ~ 6GB

- The more samples are needed per batch, the more efficient is the WebDataset format (fewer I/Os).



Test sur Imagenet.

In WebDataset format on128 shards, Imagenet weighs 826GB (versus 153GB for the jpeg version).

When increasing the batch size, we increase the number of I/O requests.

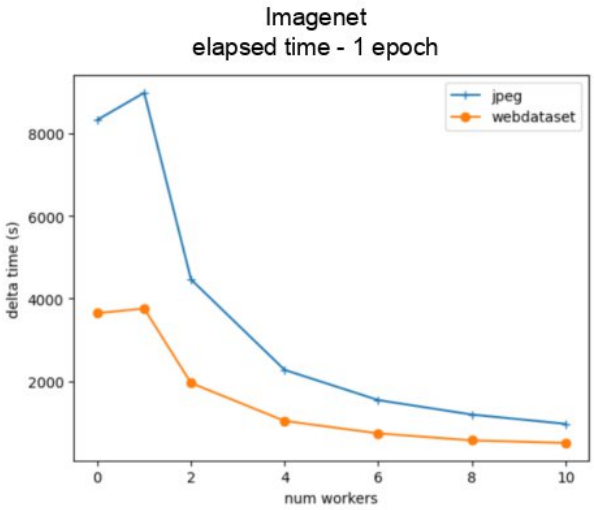
We see that with the WebDataset format, a decrease in the number of I/Os optimizes the code speed.

WebDataset - Perfomance test

I/O loop over the dataset
(calculation-free iterations)

Imagenet ~ 1.3M images
128 shards ~10k images per shard (+labels)
1 shard (images + labels) ~ 2GB

- The WebDataset format scales up.



41

We increase the number of workers to verify the scalability of the WebDataset version.

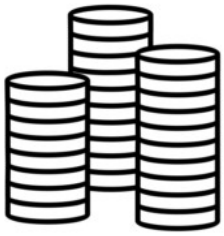
Conclusion on dataset format

- The more inodes your dataset contains, the more IOs you will request during training, and the more difficulty the parallel file system will have
- For large datasets, gather several samples in archives to minimize the number of inodes
- Standard formats already exist like WebDataset (tar files)

44

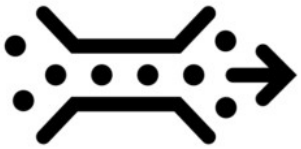
Conclusion

Storage disks



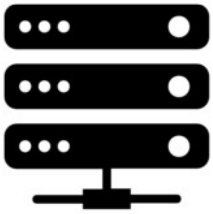
1. I/O performance

Interconnection network
Omnipath/InfiniBand



2. Shared bandwidth

CPU workers



3. Decoder performance

- Disk spaces: WORK or SCRATCH ?
- Data format: binary or compressed ?
- Dataset format: WebDataset format ?